



Tec Gurus
SOMOS Y FORMAMOS EXPERTOS EN T.I.

CURSO

Linux Desde Cero

CLASE 3

I/O, procesos y Bash

www.tecgurus.net

INSTRUCTOR

Ing. Abimael Domínguez



AGENDA • 09:00–14:00

Clase 3 — I/O, procesos y Bash

Archivos: capítulos 2, 8, 9, 10 y ejercicios seleccionados de 12.

HORA	ACTIVIDAD
09:00–09:15	Buscar un patrón y guardar resultado.
09:15–10:05	<code>stdin</code> , <code>stdout</code> , <code>stderr</code> y redirecciones.
10:05–11:00	Pipes, <code>tee</code> , filtros y CSV.
11:00–11:30	Receso.
11:30–12:25	Procesos, jobs, señales, <code>nohup</code> , <code>time</code> , servicios y journal.
12:25–13:10	Shebang, variables, argumentos y estados.
13:10–13:50	Crear y probar el resumen de logs.
13:50–14:00	Evidencia y cierre.

No recortar: descriptores, pipes, `kill`, argumentos y validación.

Recortar primero: prioridad y detalles de `systemd`.

Índice interactivo

Selecciona un apartado para ir directamente a su contenido. En cada encabezado encontrarás el enlace para volver aquí.

CAPÍTULO 2	
Entrada y salida del sistema	
Índice	→
Objetivos	→
Antes de empezar	→
2.1 Entrada y salida	→
Ejemplo resuelto	→
Terminales TTY y PTY	→
- Diferencia entre terminal y shell - Cómo identificar cada uno	→
Todo es componible	→
Dispositivos útiles	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 2 — Flujo de componentes	→
Criterios de comprobación	→
Checklist	→

CAPÍTULO 8	
Redirecciones y tuberías	
Índice	→
Objetivos	→
Antes de empezar	→
8.1 Redirecciones	→
Orden de redirecciones	→
8.2 Tuberías	→
8.3 Bifurcación con tee	→
8.4 Redirección de errores	→
Práctica guiada resuelta — Reporte de incidentes	→
Errores frecuentes	→
Reto 8 — Pipeline auditable	→
Criterios de comprobación	→
Checklist	→

[Capítulos 9, 10 y selección de 12 →](#)

Índice interactivo

Procesos, servicios, programas de comandos y práctica pertinente del taller ampliado.

CAPÍTULO 9	
Procesos y servicios	
Índice	→
Objetivos	→
Antes de empezar	→
Inspección con ps, pgrep y top	→
9.1 Background, señales, prioridad y nohup	→
Trabajos del shell	→
Señales y kill	→
Prioridad	→
Persistencia con nohup	→
9.2 Medición con time	→
9.3 top y recursos	→
Servicios y logs con systemd	→
Práctica guiada resuelta — Ciclo de un proceso	→
Errores frecuentes	→
Reto 9 — Operación observable	→
Criterios de comprobación	→
Checklist	→

CAPÍTULO 10	
Programas de comandos con Bash	
Índice	→
Objetivos	→
Antes de empezar	→
10.1 Comentarios y shebang	→
10.2 Variables y comillas	→
10.3 echo y printf	→
10.4 Parámetros y estado	→
Script resuelto: resumen-log.sh	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 10 — Resumen parametrizado	→
Criterios de comprobación	→
Checklist	→

CAPÍTULO 12 · SELECCIÓN	
Ejercicios de Bash pertinentes	
Ejercicios seleccionados del taller	→
Cómo leer los argumentos de los scripts	→
3.2 Argumentos	→
3.3 Búsqueda parametrizada	→
6.2 Clasificar un log	→



MODELO DE DATOS

Entrada y salida del sistema

Descriptores estándar, terminales, shell y composición de herramientas mediante flujos de texto.

Capítulo 2

stdin · stdout · stderr

TTY · PTY

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [2.1 Entrada y salida](#)
- [Terminales TTY y PTY](#)
- [Todo es componible](#)
- [Dispositivos útiles](#)
- [Práctica guiada resuelta](#)
- [Errores frecuentes](#)
- [Reto 2](#)

Objetivos

[Índice ↑](#)

- Reconocer entrada estándar, salida estándar y salida de error.
- Relacionar un proceso con su terminal.
- Comprender por qué los comandos pequeños pueden componerse.

Antes de empezar

[Índice ↑](#)

Ejecuta los ejemplos desde la raíz del curso. Este capítulo sólo creará archivos dentro de `laboratorio/salida`:

```
cd ~/linux-desde-cero
mkdir -p laboratorio/salida
```

TERMINAL · BASH

`laboratorio` es una carpeta de práctica dentro del repositorio; no es una parte especial de Linux. Puedes eliminarla y volverla a crear sin afectar `data/` ni los capítulos.

Continuidad entre sesiones. Puedes llegar con evidencia de las Clases 1 o 2 dentro de `laboratorio/`. Este capítulo sólo necesita `laboratorio/salida`, que `mkdir -p` crea sin borrar las demás carpetas. Si `data/dummy_logs.txt` no existe cuando un ejemplo lo solicite, no crees un archivo vacío: confirma que estás en `~/linux-desde-cero` y recupera el repositorio con el instructor.

Los operadores completos se practican en `08-redirecciones-y-tuberias.md`. Este capítulo establece el modelo mental.

2.1 Entrada y salida

[Índice ↑](#)

Al iniciar un proceso, Linux le proporciona tres descriptores:

Descriptor	Nombre	Uso
0	<code>stdin</code>	Entrada de datos.
1	<code>stdout</code>	Resultado normal.
2	<code>stderr</code>	Diagnósticos y errores.

SALIDA REPRESENTATIVA

```
teclado/archivo → stdin → proceso → stdout → terminal/archivo
                  ↳ stderr → terminal/archivo
```

Ejemplo resuelto

[Índice ↑](#)

Los nombres `modelo-A` y `modelo-B` son texto de ejemplo. `/etc/hosts` es un archivo real del sistema y `/ruta/inexistente` es una ruta deliberadamente falsa para provocar un mensaje en `stderr`.

TERMINAL · BASH

```
printf '%s\n' "modelo-A" "modelo-B"
ls /etc/hosts
ls /ruta/inexistente
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
modelo-A
modelo-B
/etc/hosts
ls: cannot access '/ruta/inexistente': No such file or directory
```

Los dos primeros comandos producen salida normal. El tercero escribe el diagnóstico en `stderr`; esto permite separar datos válidos de errores.

Terminales TTY y PTY

[Índice ↑](#)

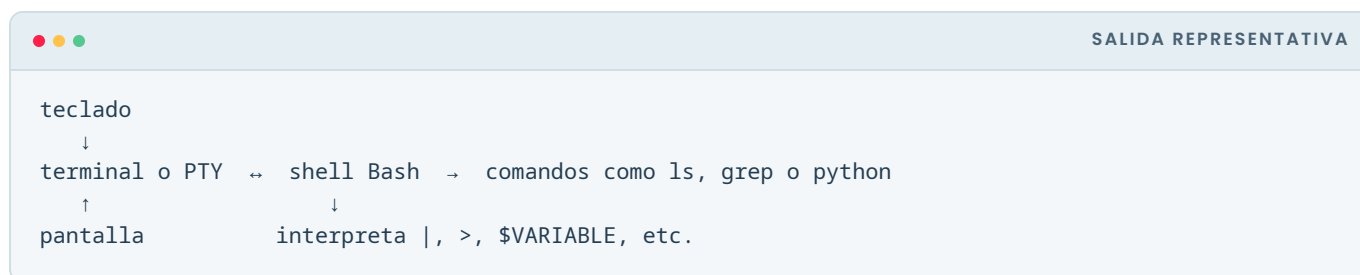
- **TTY:** nombre histórico de una terminal.
- **PTY:** pseudoterminal usado por SSH y emuladores gráficos.

Diferencia entre terminal y shell

[Índice ↑](#)

Una **terminal** es el medio de entrada y salida: recibe lo que escribes, muestra texto y proporciona funciones como tamaño de pantalla, colores y combinaciones de teclas. GNOME Terminal, Konsole y Windows Terminal son emuladores de terminal.

Un **shell** es el programa que interpreta lo escrito dentro de esa terminal. Reconoce comandos, variables, pipes, redirecciones y scripts. Bash, Zsh, Fish y PowerShell son shells.



Una terminal puede ejecutar distintos shells. Del mismo modo, un shell puede ejecutarse sin una ventana gráfica: por ejemplo, dentro de un script automatizado. En una conexión SSH normalmente ocurre esto:

1. SSH crea una pseudoterminal remota o PTY.
2. Dentro de esa PTY inicia el shell del usuario, por ejemplo Bash.
3. Bash interpreta los comandos y la PTY devuelve la salida a tu terminal local.

Cómo identificar cada uno

[Índice ↑](#)

```
tty
ps -p "$$" -o comm=
echo "$SHELL"
ps -o pid,ppid,TTY,stat,comm -p "$$"
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
/dev/pts/0
bash
/bin/bash
  PID  PPID  TT      STAT COMMAND
 2481  2479 pts/0   Ss    bash
```

- `tty` muestra la terminal asociada: aquí es `/dev/pts/0`.
- `ps -p "$$" -o comm=` muestra el shell que está ejecutándose ahora: `bash`.
- `$SHELL` muestra el shell configurado como predeterminado para el usuario. Puede ser distinto del shell activo si abriste otro manualmente.

Si `tty` responde `not a tty`, el comando se ejecutó sin terminal interactiva —por ejemplo, desde automatización o algunos entornos de ejecución—. El shell puede seguir existiendo aunque no tenga una TTY asociada.

- `PID`: identificador del proceso.
- `PPID`: proceso padre.
- `TT`: terminal asociada.
- `STAT`: estado del proceso.
- `COMMAND`: programa ejecutado.

Todo es componible

[Índice ↑](#)

Muchas herramientas aceptan texto por entrada y producen texto por salida. Esto permite reutilizarlas:

```
printf '%s\n' api worker api scheduler | sort | uniq -c
```

Salida esperada:

```
2 api
1 scheduler
1 worker
```

- `sort` agrupa líneas iguales.
- `uniq -c` cuenta elementos consecutivos; por eso se ordena primero.
- Cada comando resuelve una parte y no necesita conocer al siguiente.

Dispositivos útiles

[Índice ↑](#)

TERMINAL · BASH

```
printf 'mensaje visible\n' > /dev/tty  
printf 'mensaje descartado\n' > /dev/null
```

- `/dev/tty`: terminal del proceso.
- `/dev/null`: descarta lo que recibe.
- Los dispositivos se exponen mediante archivos especiales, pero no son archivos regulares.

Práctica guiada resuelta

[Índice ↑](#)

Crearemos `componentes.txt` con cuatro líneas y luego contaremos cuántas veces aparece cada componente. No necesitas crear el archivo manualmente: el primer `printf` lo hace.

```
mkdir -p laboratorio/salida

printf '%s\n' inference training inference serving > laboratorio/salida/componentes.txt
sort laboratorio/salida/componentes.txt | uniq -c

tty
ps -o pid,ppid,ttty,stat,comm -p "$$"
```

Resultado: se crea una entrada reproducible, se cuenta cada componente y se identifica el shell que ejecutó el pipeline.

Comprueba los archivos:

```
wc -l laboratorio/salida/componentes.txt
cat laboratorio/salida/componentes.txt
```

La primera salida debe indicar `4`, porque escribimos cuatro componentes.

Errores frecuentes

[Índice ↑](#)

- Creer que todo texto visible pertenece a `stdout`.
- Usar `uniq -c` sin ordenar cuando se quieren totales globales.
- Confundir una terminal con el shell: la terminal transporta y muestra texto; el shell interpreta comandos dentro de ella.

Reto 2 — Flujo de componentes

[Índice ↑](#)[Ver respuesta](#)

Crea `laboratorio/reto2/servicios.txt` con al menos ocho nombres de servicios repetidos. Genera `laboratorio/reto2/resumen.txt` con el conteo de cada servicio, ordenado por nombre. Debes conservar ambos archivos.

Criterios de comprobación

[Índice ↑](#)

- No se cuenta manualmente.
- Entrada y resumen están en archivos distintos.
- La suma de los conteos coincide con el número de líneas de entrada.

Checklist

[Índice ↑](#)

- ☐ Identifico los descriptores 0, 1 y 2.
- ☐ Puedo explicar TTY frente a shell.
- ☐ Comprendo por qué `sort | uniq -c` funciona.



02

COMPOSICIÓN EN TERMINAL

Redirecciones y tuberías

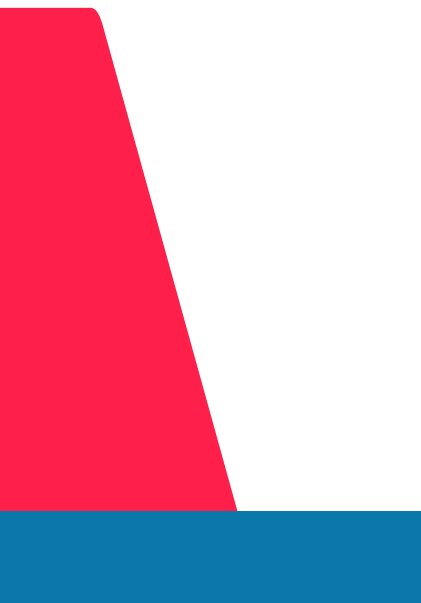
Separar datos de errores, conectar procesos y construir reportes observables con pipes y tee.

Capítulo 8

> · >> · 2>

pipe

tee



Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [8.1 Redirecciones](#)
- [8.2 Tuberías](#)
- [8.3 Bifurcación con `tee`](#)
- [8.4 Redirección de errores](#)
- [Práctica guiada: reporte de incidentes](#)
- [Errores frecuentes](#)
- [Reto 8](#)

Objetivos

[Índice ↑](#)

- Redirigir `stdout` y `stderr` por separado o juntos.
- Construir pipelines legibles.
- Duplicar una salida con `tee`.
- Producir reportes sin perder diagnósticos.

Antes de empezar

[Índice ↑](#)

Ejecuta todo desde la raíz del curso. Prepararemos `laboratorio/io` y usaremos `data/dummy_logs.txt` como entrada; los archivos originales de `data/` no se modifican.

```
cd ~/linux-desde-cero
mkdir -p laboratorio/io
```

Continuidad entre sesiones. `laboratorio/io` es exclusivo de esta parte de la Clase 3 y puede crearse aunque conserves `laboratorio/shell` o evidencia de la Clase 2. `data/dummy_logs.txt` es un archivo versionado de entrada: compruébalo con `ls data/dummy_logs.txt`; si falta, no lo sustituyas por un archivo vacío. Verifica primero `pwd` y recupera el repositorio si fuera necesario.

8.1 Redirecciones

[Índice ↑](#)

Operador	Acción
>	crea/sobrescribe la salida
>>	agrega al final
<	toma entrada desde archivo
2>	redirige errores
2>&1	envía descriptor 2 al destino actual de 1

TERMINAL · BASH

```
mkdir -p laboratorio/io
printf '%s\n' alpha beta gamma > laboratorio/io/entrada.txt
wc -l < laboratorio/io/entrada.txt > laboratorio/io/conteo.txt
ls /etc/hosts /ruta-inexistente \
  > laboratorio/io/encontrados.txt \
  2> laboratorio/io/errores.txt
```

`/etc/hosts` existe y debe quedar en `encontrados.txt`. `/ruta-inexistente` es falsa a propósito y su mensaje debe quedar en `errores.txt`. Aunque `ls` termina con error parcial, ambos archivos se crean.

Compruébalo:

TERMINAL · BASH

```
cat laboratorio/io/encontrados.txt
cat laboratorio/io/errores.txt
```

> elimina el contenido previo. Usa >> cuando realmente deseas acumular.

Orden de redirecciones

[Índice ↑](#)

TERMINAL · BASH

```
comando > salida.log 2>&1
```

Primero conecta `stdout` al archivo y luego conecta `stderr` al mismo destino. El orden puede cambiar el resultado.

`comando` y `salida.log` son marcadores conceptuales. Un ejemplo resuelto y seguro es:

TERMINAL · BASH

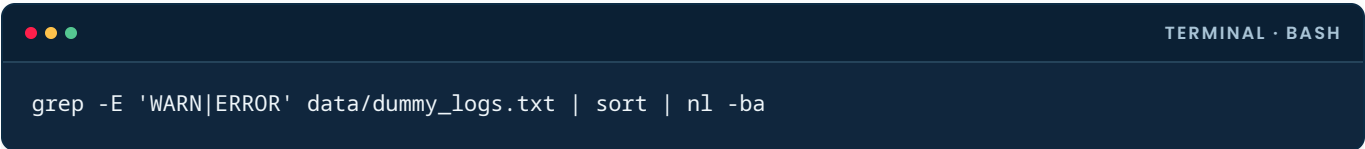
```
ls /etc/hosts /ruta-inexistente > laboratorio/io/salida-completa.log 2>&1 || true  
cat laboratorio/io/salida-completa.log
```

`|| true` se usa aquí porque el error es intencional; no debe añadirse indiscriminadamente a comandos reales.

8.2 Tuberías

[Índice ↑](#)

Una tubería conecta `stdout` del comando izquierdo con `stdin` del derecho:

A terminal window with a dark blue background and a title bar that says "TERMINAL · BASH". The command `grep -E 'WARN|ERROR' data/dummy_logs.txt | sort | nl -ba` is entered in the terminal.

```
grep -E 'WARN|ERROR' data/dummy_logs.txt | sort | nl -ba
```

- `|`: conecta procesos.
- `nl -ba`: numera incluso líneas vacías.
- Cada etapa debe poder verificarse de forma independiente.

El estado normal de un pipeline corresponde al último comando. En scripts robustos se usa `set -o pipefail` para detectar fallos anteriores.

8.3 Bifurcación con tee

[Índice ↑](#)

TERMINAL · BASH

```
grep -E 'WARN|ERROR' data/dummy_logs.txt \  
| tee laboratorio/io/incidentes.txt \  
| wc -l
```

`tee` copia la entrada al archivo y también la envía al siguiente comando. `tee -a` agrega en lugar de sobrescribir.

8.4 Redirección de errores

[Índice ↑](#)

TERMINAL · BASH

```
find /etc -type f -name '*.conf' \  
> laboratorio/io/configs.txt \  
2> laboratorio/io/find-errors.txt
```

No descartes errores con `/dev/null` hasta entenderlos. Durante aprendizaje y automatización, conservar diagnósticos facilita depurar.

Práctica guiada resuelta — Reporte de incidentes

[Índice ↑](#)

La práctica crea `reporte.txt` y `reporte-errors.log`. No necesitas crear ninguno antes; las redirecciones y `tee` lo hacen.

TERMINAL · BASH

```
mkdir -p laboratorio/io

{
  printf 'REPORTE DE INCIDENTES\n'
  printf 'Generado: %s\n\n' "$(date --iso-8601=seconds)"
  grep -E 'WARN|ERROR' data/dummy_logs.txt
} 2> laboratorio/io/reporte-errors.log \
| tee laboratorio/io/reporte.txt

printf 'Total: '
grep -Ec 'WARN|ERROR' laboratorio/io/reporte.txt
```

Explicación:

- `{ ...; }` agrupa salidas.
- La fecha se calcula al ejecutar.
- `stderr` queda separado en `reporte-errors.log`.
- `tee` permite ver y guardar.
- `grep -c` cuenta líneas coincidentes.

La salida final debe mostrar `Total: 3`, porque el fixture contiene dos `ERROR` y un `WARN`.

Errores frecuentes

[Índice ↑](#)

- Sobrescribir un archivo con `>` cuando se quería `>>`.
- Redirigir al mismo archivo que todavía se está leyendo.
- Crear pipelines largos sin probar cada segmento.
- Ocultar `stderr` y confundir una salida vacía con éxito.

Reto 8 — Pipeline auditable

[Índice ↑](#)[Ver respuesta](#)

Crea `laboratorio/reto8/usuarios.txt` con los usuarios de `data/dummy_logs.txt`, ordenados y sin duplicados. Guarda errores en `laboratorio/reto8/errores.log` y muestra en terminal el número de usuarios únicos sin releer manualmente el archivo original.

Criterios de comprobación

[Índice ↑](#)

- Usa al menos una tubería y `tee`.
- Datos y errores están separados.
- El archivo final contiene sólo nombres únicos.
- El conteo coincide con las líneas del archivo final.

Checklist

[Índice ↑](#)

- ☐ Distingo `>`, `>>`, `2>` y `2>&1`.
- ☐ Construyo y pruebo pipelines por etapas.
- ☐ Uso `tee` para observar y guardar.

03

OPERACIÓN OBSERVABLE

Procesos, señales y servicios

Inspección de procesos, control de jobs, señales graduales, medición y consulta segura de systemd.

Capítulo 9

ps · jobs

kill

systemctl · journalctl

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [Inspección con `ps`, `pgrep` y `top`](#)
- [9.1 Background, señales, prioridad y `nohup`](#)
- [9.2 Medición con `time`](#)
- [9.3 `top` y recursos](#)
- [Servicios y logs con `systemd`](#)
- [Práctica guiada: ciclo de un proceso](#)
- [Errores frecuentes](#)
- [Reto 9](#)

Objetivos

[Índice ↑](#)

- Identificar procesos y relaciones padre/hijo.
- Controlar trabajos en foreground y background.
- Enviar señales y ajustar prioridad.
- inspeccionar servicios y logs con systemd.

Antes de empezar

[Índice ↑](#)

Abre una terminal nueva, sitúate en la raíz del curso y crea la carpeta de salida:

```
cd ~/linux-desde-cero
mkdir -p laboratorio
```

Trabajaremos únicamente con procesos que tú mismo inicies (`script_largo.sh` y `sleep`). No copies un PID de `top` para terminar procesos del sistema.

Continuidad entre sesiones. Esta práctica reutiliza la raíz del curso, no los archivos creados en la Clase 2. Antes de iniciar, comprueba `ls script_largo.sh data/dummy_logs.txt`. Si falta `script_largo.sh` o el log, no inventes un sustituto: vuelve a la raíz con `cd ~/linux-desde-cero` y pide recuperar el repositorio. `mkdir -p laboratorio` conserva las demás evidencias de laboratorio.

Inspección con `ps`, `pgrep` y `top`

[Índice ↑](#)

TERMINAL · BASH

```
ps -o pid,ppid,user,stat,%cpu,%mem,comm -p "$$"
ps aux --sort=-%cpu | head
pgrep -a bash
top
```

- `PID` / `PPID` : proceso y padre.
- `STAT` : estado; `R` ejecutando, `S` esperando, `T` detenido, `Z` zombie.
- `pgrep -a` : PID y línea de comando.
- En `top`, presiona `P` para CPU, `M` para memoria y `q` para salir.

9.1 Background, señales, prioridad y `nohup`

[Índice ↑](#)

Trabajos del shell

[Índice ↑](#)

`script_largo.sh` imprime una línea por segundo durante aproximadamente un minuto. `&` lo inicia sin bloquear la terminal.

```
TERMINAL · BASH

bash script_largo.sh > laboratorio/proceso.log 2>&1 &
jobs -l
fg %1
```

`%1` significa “job número 1 de esta terminal”, no PID 1. Confirma el número real en la primera columna de `jobs -l`; si aparece `[2]`, usa `fg %2`.

Durante `fg`, presiona `Ctrl+Z` para suspender y luego:

```
TERMINAL · BASH

bg %1
jobs -l
```

- `&`: inicia en background.
- `jobs`: trabajos del shell actual, no todos los procesos.
- `fg` / `bg`: reanuda en foreground/background.
- `Ctrl+Z`: suspende; no termina.

Señales y `kill`

[Índice ↑](#)

En este bloque, `sleep 300 &` crea el proceso y `#!` entrega exactamente su PID. La variable `pid` no existe antes de la segunda línea; nosotros la definimos con `pid=#!`.

```
TERMINAL · BASH

sleep 300 &
pid=#!
kill -TERM "$pid"
wait "$pid" 2>/dev/null || true
```

- `#!`: PID del último proceso en background.
- `TERM` solicita cierre ordenado y es la primera opción.
- `KILL` no puede manejarse ni limpiar recursos; úsala sólo si `TERM` no funciona.
- `wait`: espera y recoge el estado del proceso hijo.

Prioridad

[Índice ↑](#)

Volvemos a crear un `sleep` independiente. La variable `pid` se reemplaza con el PID nuevo, por eso `renice` y `kill` sólo afectan este proceso de práctica.

```

nice -n 10 sleep 60 &
pid=$!
ps -o pid,ni,comm -p "$pid"
renice 15 -p "$pid"
kill "$pid"

```

Un valor nice mayor concede menos prioridad de CPU. Aumentar prioridad suele requerir privilegios.

Persistencia con `nohup`

[Índice ↑](#)

```

nohup bash script_largo.sh > laboratorio/nohup.log 2>&1 &
echo "$!" > laboratorio/nohup.pid

```

`nohup` ignora SIGHUP, pero no convierte el proceso en servicio. Para cargas operativas se prefiere un supervisor o unidad systemd.

El comando guarda el PID en `laboratorio/nohup.pid`. Al terminar la demostración, limpia el proceso de forma controlada:

```

pid=$(cat laboratorio/nohup.pid)
if kill -0 "$pid" 2>/dev/null; then
    kill -TERM "$pid"
else
    echo "El proceso ya había terminado"
fi
wait "$pid" 2>/dev/null || true

```

`kill -0` no termina el proceso: sólo comprueba si existe y si tienes permiso para señalarlo. `wait` sólo funciona si sigue siendo hijo del shell actual; el `|| true` tolera que ya haya terminado.

9.2 Medición con `time`

[Índice ↑](#)

TERMINAL · BASH

```
time grep -c 'ERROR' data/dummy_logs.txt
```

- `real` : tiempo de pared.
- `user` : CPU en espacio de usuario.
- `sys` : CPU dentro del kernel.

9.3 top y recursos

[Índice ↑](#)

TERMINAL · BASH

```
free -h  
uptime  
df -h /
```

La carga no es un porcentaje. Debe interpretarse con cantidad de CPU, procesos bloqueados y contexto del sistema.

Servicios y logs con systemd

[Índice ↑](#)

En Ubuntu 24.04:



TERMINAL · BASH

```
systemctl status ssh --no-pager
systemctl is-enabled ssh
journalctl -u ssh --since '30 minutes ago' --no-pager
```

- `status` : estado y mensajes recientes.
- `is-enabled` : si inicia automáticamente; no indica necesariamente que esté activo.
- `journalctl -u` : filtra una unidad.
- `--since` : limita el periodo.
- `--no-pager` : salida no interactiva, útil en scripts y clases.

Si aparece `Unit ssh.service could not be found`, el servidor OpenSSH no está instalado en ese equipo. En la EC2 del curso debe existir porque es el servicio que permite la conexión inicial.

No reinicies servicios compartidos sin autorización.

Práctica guiada resuelta — Ciclo de un proceso

[Índice ↑](#)

No debes sustituir `$pid` manualmente. La línea `pid=$!` captura el PID del `sleep` que acaba de arrancar y todas las señales posteriores usan ese mismo valor.

TERMINAL · BASH

```
mkdir -p laboratorio
sleep 120 &
pid=$!
printf 'PID=%s\n' "$pid" | tee laboratorio/proceso.pid
ps -o pid,ppid,stat,ni,etime,comm -p "$pid"
kill -STOP "$pid"
ps -o pid,stat,comm -p "$pid"
kill -CONT "$pid"
kill -TERM "$pid"
wait "$pid" 2>/dev/null || true
ps -p "$pid" || printf 'Proceso finalizado\n'
```

`STOP` suspende, `CONT` reanuda y `TERM` solicita terminar. La práctica usa un proceso creado por el propio alumno.

La columna `STAT` debe contener `T` después de `STOP`. Al final se imprime `Proceso finalizado`.

Errores frecuentes

[Índice ↑](#)

- Ejecutar `kill -9` como primera respuesta.
- Confundir PID con número de job `%1`.
- Cerrar la terminal creyendo que `&` garantiza persistencia.
- Reiniciar un servicio sin revisar logs.

Reto 9 — Operación observable

[Índice ↑](#)

Ver respuesta

Trabaja dentro de `laboratorio/reto9`. Ejecuta `script_largo.sh` en background, guarda salida/errores en `proceso.log` y el PID en `proceso.pid`. Demuestra que está activo, mide otra ejecución corta y termina el proceso limpiamente. Guarda el estado del servicio SSH en `ssh-status.txt`.

Criterios de comprobación

[Índice ↑](#)

- PID y log se conservan en archivos.
- Se usa `TERM` antes de cualquier señal forzada.
- El reporte distingue proceso, job y servicio.
- Al final el PID ya no existe.

Checklist

[Índice ↑](#)

- ☐ Identifico PID, PPID, estado y prioridad.
- ☐ Controlo jobs con `jobs`, `fg` y `bg`.
- ☐ Uso señales de manera gradual.
- ☐ Consulto servicios y journal sin modificarlos.

04

AUTOMATIZACIÓN CON BASH

Programas de comandos

Shebang, variables, argumentos, validación y estados de salida aplicados a un resumen reproducible de logs.

Capítulo 10

`#!/usr/bin/env bash`

`## · $1 · "$@"`

`exit`

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [10.1 Comentarios y shebang](#)
- [10.2 Variables y comillas](#)
- [10.3 `echo` y `printf`](#)
- [10.4 Parámetros y estado](#)
- [Práctica guiada resuelta](#)
- [Errores frecuentes](#)
- [Reto 10](#)

Objetivos

[Índice ↑](#)

- Crear y ejecutar un script Bash.
- Usar comentarios, variables, argumentos y códigos de salida.
- Validar entradas antes de procesarlas.
- Generar un reporte útil para operación.

Antes de empezar

[Índice ↑](#)

Sitúate en la raíz del curso y prepara el laboratorio:

Continuidad entre sesiones. `preparar-lab.sh` crea una copia limpia de fixtures dentro de `laboratorio/data`, pero también elimina y recrea toda la carpeta `laboratorio/`. Ejecútalo sólo si el grupo decidió reiniciar el laboratorio y ya guardó la evidencia de las Clases 1 y 2. Si quieres conservarla, no ejecutes el script; crea sólo `laboratorio/scripts` y usa los datos versionados que indique cada ejercicio.



TERMINAL · BASH

```
cd ~/linux-desde-cero
bash ejercicios-bash-scripting/preparar-lab.sh
mkdir -p laboratorio/scripts
```


10.1 Comentarios y shebang

[Índice ↑](#)

TERMINAL · BASH

```
#!/usr/bin/env bash
# Resume niveles de un archivo de log.
```

- El shebang elige el intérprete cuando el archivo se ejecuta directamente.
- Un comentario explica intención o una decisión; no debe repetir cada línea.

Las dos líneas anteriores son el contenido inicial de un script, no comandos independientes. Para crear un ejemplo completo listo para ejecutar:



TERMINAL · BASH

```
printf '%s\n' \  
  '#!/usr/bin/env bash' \  
  '# Primer script del curso' \  
  'echo "Hola desde Bash"' \  
  > laboratorio/scripts/hola.sh  
chmod +x laboratorio/scripts/hola.sh  
laboratorio/scripts/hola.sh
```

Salida esperada: `Hola desde Bash`.

10.2 Variables y comillas

[Índice ↑](#)

TERMINAL · BASH

```
nombre="reporte diario"
fecha=$(date +%F)
printf 'Nombre: %s | Fecha: %s\n' "$nombre" "$fecha"
```

- `variable=valor`: no admite espacios alrededor de `=`.
- `${...}`: sustitución de comando.
- `"$variable"`: evita separación por espacios y expansión de globs.
- Las mayúsculas suelen reservarse para variables de entorno.

10.3 echo y printf

[Índice ↑](#)

`echo` es cómodo para mensajes simples. `printf` es más predecible para formatos:



TERMINAL · BASH

```
printf 'usuario=%s errores=%d\n' "$USER" 2
```

`%s` recibe texto y `%d` un entero. Los argumentos se sustituyen en orden.

10.4 Parámetros y estado

[Índice ↑](#)

Expresión	Significado
\$0	nombre usado para ejecutar
\$1 ...	argumentos posicionales
\$#	cantidad de argumentos
"\$@"	todos, conservando límites
\$?	estado del último comando

Convención: 0 significa éxito; otro valor indica que no se completó lo solicitado.

Script resuelto: `resumen-log.sh`

[Índice ↑](#)

El bloque siguiente es el contenido completo del script. En el repositorio ya existe una versión equivalente en `ejercicios-bash-scripting/filtro_logs.sh`; no necesitas volver a teclearla para ejecutar la práctica.

```

#!/usr/bin/env bash
set -o nounset
set -o pipefail

if [[ $# -ne 1 ]]; then
    printf 'Uso: %s <archivo-log>\n' "$0" >&2
    exit 64
fi

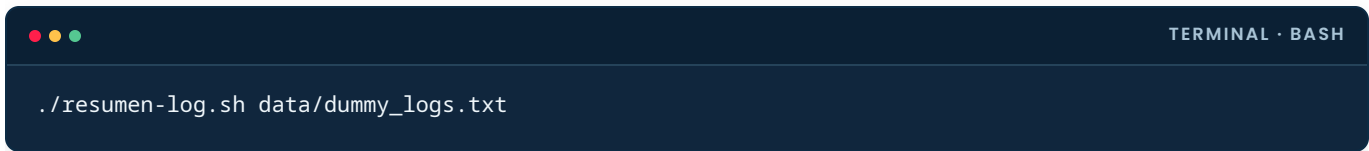
log=$1
if [[ ! -r "$log" ]]; then
    printf 'Error: no puedo leer %s\n' "$log" >&2
    exit 66
fi

printf 'Archivo: %s\n' "$log"
for nivel in INFO WARN ERROR; do
    total=$(grep -c "$nivel" "$log" || true)
    printf '%-5s %d\n' "$nivel" "$total"
done

```

- `[[ ... ]]`: prueba de Bash.
- `-r`: archivo legible.
- `>&2`: mensaje hacia `stderr`.
- `exit`: termina con un estado explícito.
- `grep -c` puede devolver 1 cuando no encuentra coincidencias; `|| true` permite reportar cero.

Cuando se ejecuta así:



```
./resumen-log.sh data/dummy_logs.txt
```

los valores son:

- `$0` → `./resumen-log.sh`;
- `$1` → `data/dummy_logs.txt`;
- `$#` → `1`.

Por eso `log=$1` guarda la ruta del archivo en la variable `log`.

Práctica guiada resuelta

[Índice ↑](#)

Primero copiamos el script existente al laboratorio y lo renombramos `resumen-log.sh`. Después hacemos dos pruebas: una válida con `data/dummy_logs.txt` y otra inválida con `archivo-inexistente`.

```
TERMINAL · BASH

mkdir -p laboratorio/scripts
cp ejercicios-bash-scripting/filtra_logs.sh laboratorio/scripts/resumen-log.sh
chmod +x laboratorio/scripts/resumen-log.sh

laboratorio/scripts/resumen-log.sh data/dummy_logs.txt \
> laboratorio/scripts/reporte.txt
estado_ok=?

laboratorio/scripts/resumen-log.sh archivo-inexistente \
2> laboratorio/scripts/error.txt || estado_error=?

printf 'Éxito=%s Error=%s\n' "$estado_ok" "${estado_error:-0}"
cat laboratorio/scripts/reporte.txt
cat laboratorio/scripts/error.txt
```

La prueba cubre camino exitoso y fallo esperado, preservando ambas salidas.

`estado_ok` debe ser `0`. `estado_error` debe ser distinto de cero —en este script es `66`— y `error.txt` debe explicar que el archivo no puede leerse.

Errores frecuentes

[Índice ↑](#)

- Ejecutar datos como código con `eval`.
- Usar `$*` cuando se necesitan argumentos separados.
- Omitir comillas en rutas.
- Mostrar un error pero finalizar con código 0.
- Activar opciones estrictas sin comprender comandos que legítimamente devuelven estado distinto de cero.

Reto 10 — Resumen parametrizado

[Índice ↑](#)[Ver respuesta](#)

Escribe `laboratorio/scripts/reto10.sh`. Debe recibir dos argumentos: `<archivo_log>` y `<archivo_reporte>`. Debe validar ambos, generar conteos por nivel, listar usuarios únicos y no crear el reporte si la entrada es inválida.

Criterios de comprobación

[Índice ↑](#)

- Funciona con rutas que contienen espacios.
- Mensajes de error van a `stderr`.
- Devuelve 0 al completar y otro código al fallar.
- No contiene una ruta fija a `data/dummy_logs.txt`.

Checklist

[Índice ↑](#)

- ☐ Escribo shebang y comentarios útiles.
- ☐ Cito variables y uso "\$@" correctamente.
- ☐ Valido argumentos y archivos.
- ☐ Produzco estados de salida verificables.



05

PRÁCTICA SELECCIONADA

Argumentos y logs

Ejercicios puntuales del taller ampliado para reforzar límites de argumentos, búsquedas parametrizadas y validación de logs.

Capítulo 12 · selección

argumentos

grep

logs

Ejercicios seleccionados del taller

[Índice ↑](#)

Esta selección refuerza argumentos, validación y procesamiento de logs sin incorporar los temas ampliados que exceden la sesión.

Cómo leer los argumentos de los scripts

[Índice ↑](#)

Un argumento es un valor escrito después del nombre del script. Por ejemplo:

```
./ejercicios-bash-scripting/01_contar_ganadores.sh data/basketball_scores.csv
```

En esa ejecución:

- `$0` es la ruta del script;
- `$1` es `data/basketball_scores.csv`;
- `$#` vale `1` porque se proporcionó un argumento.

3.2 Argumentos

[Índice ↑](#)

```
./ejercicios-bash-scripting/05_argumentos.sh "hola mundo" Linux
```

Salida esperada:

```
Primer argumento: hola mundo
Segundo argumento: Linux
Cantidad: 2
Todos: [hola mundo] [Linux]
```

- `$1`, `$2`: argumentos posicionales.
- `$#`: cantidad.
- `"$@"`: conserva cada argumento, incluso con espacios.

3.3 Búsqueda parametrizada

[Índice ↑](#)

TERMINAL · BASH

```
../ejercicios-bash-scripting/06_grep_parametrizado.sh \  
Engineer data/hire_data salida/engineer.csv  
head salida/engineer.csv
```

El script valida directorio, patrón y destino; usa `grep -h` para no anteponer nombres de archivo.

6.2 Clasificar un log

[Índice ↑](#)

TERMINAL · BASH

```
../ejercicios-bash-scripting/13_mover_logs.sh \  
data/logs/servicio.log data/good_logs
```

Se mueve únicamente cuando aparecen ambos patrones `SRVM_` y `vpt`. `grep -q` comprueba sin imprimir.



EVIDENCIA Y CIERRE

Clase 3 completada

Al terminar la Clase 3, conserva el script, su reporte y las pruebas que demuestran tanto el camino exitoso como el error esperado.



Distingo stdin, stdout y stderr, y construyo pipelines auditables.



Controlo procesos propios con jobs y señales graduales.



Valido argumentos y archivos antes de generar un resumen de logs.

SIGUIENTE: CLASE 4 · REGEX, RED Y COPIAS REMOTAS